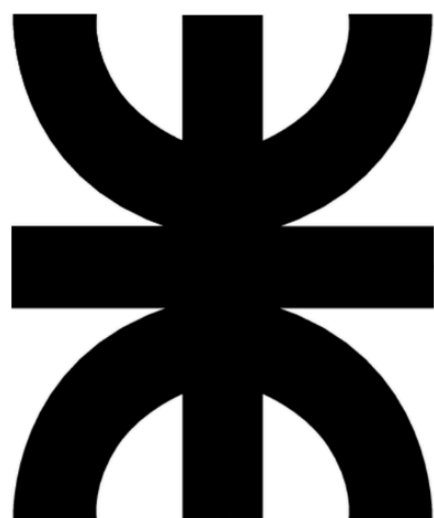




**UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL TUCUMÁN**

**VIRTUALIZACIÓN Y
CONSOLIDACIÓN DE
SERVIDORES**



**TRABAJO PRÁCTICO FINAL:
ELABORACIÓN DE BLOG PERSONAL**

Jaluf, Rocío Daniela - 52447

Índice

Principales requisitos y restricciones de desarrollo.....	3
Herramientas, recursos y servicios utilizados.....	3
Proxmox VE.....	3
Contenedores (LXC).....	3
Servidor Web Ligero: Nginx.....	4
Base de Datos: MariaDB.....	4
Paso a paso del desarrollo del blog.....	5
Creación y Configuración de la Infraestructura.....	5
Creación del contenedor de frontend:.....	6
Creación del contenedor de Base de Datos:.....	9
Trabajo con el contenedor de base de datos (CT178).....	10
Trabajo con el contenedor del frontend (CT177).....	12
Despliegue del Frontend e Integración de Servicios.....	12
Visualización del Blog desplegado.....	15
Validación de Persistencia y Gestión de Datos.....	15

Principales requisitos y restricciones de desarrollo

- El desarrollo del blog personal deberá hacerse a través de la plataforma Proxmox VE brindada por la cátedra.
- Dentro de la plataforma se deberán crear dos contenedores, uno que sirva para los servicios de interfaz gráfica de usuario y otro que funcione como servidor de base de datos.
- **Restricción estricta de recursos:** Cada contenedor deberá tener asignado un máximo absoluto de 128 MB de Memoria RAM, 1 núcleo de procesamiento y 8 GB de almacenamiento.
- **Restricción de Red:** La dirección IP de los contenedores debe construirse estáticamente utilizando el ID asignado por Proxmox en el último octeto (Formato: **172.16.90.ID/24**), con puerta de enlace y DNS apuntando a **172.16.90.1**.
- Entre las principales funcionalidades del blog se debe:
 - Listar datos personales.
 - Incorporar una foto de perfil.
 - Permitir la publicación y visualización de nuevos datos.

Herramientas, recursos y servicios utilizados

Proxmox VE

El desarrollo del blog personal se llevó a cabo por medio de Proxmox VE. Es una plataforma de virtualización, basada en sistemas de código abierto, para la implementación de máquinas virtuales y contenedores.

Brinda una fácil instalación, administración central vía web y acceso seguro.

Contenedores (LXC)

Los contenedores son como “cajas” en las que se puede meter una aplicación y todo lo que necesite para funcionar. Se pueden considerar como máquinas virtuales livianas, ya que el objetivo es el mismo, pero con la gran diferencia de que los recursos que se necesitan son mucho menores.

Esto fue vital para cumplir con el límite de 128 MB de RAM. Los recursos asignados para ambos contenedores (**43003592A** y **43003592DB**) fueron:

-Sistema operativo: Debian 12 (Standard)

-Núcleos: 1

-Memoria: 128 MB

-Almacenamiento: 8 GB

Servidor Web Ligerito: Nginx

A diferencia de servidores tradicionales y pesados, se optó por Nginx debido a su arquitectura orientada a eventos. Esto permite servir la página web consumiendo una fracción mínima de memoria RAM, garantizando que el contenedor no colapse bajo la restricción de los 128 MB.

Base de Datos: MariaDB

Se utilizó MariaDB, un sistema de administración de base de datos relacional de código abierto de MySQL. Se aplicaron configuraciones específicas para optimizar su consumo de recursos y permitir la conexión remota desde el contenedor web.

Desarrollo a Medida (PHP 8, HTML5 Y CSS3)

Para la interfaz gráfica y la lógica del blog se desarrolló una solución a medida utilizando tecnologías del ecosistema web:

- **PHP 8:** Lenguaje de scripting del lado del servidor. A diferencia de entornos como Node.js, PHP opera bajo un modelo de ejecución sin estado: el proceso se inicia únicamente al recibir una petición HTTP, genera la respuesta y termina, liberando inmediatamente los recursos de memoria. Esto garantiza un consumo de RAM mínimo y predecible, compatible con las restricciones del entorno.
- **HTML5:** Lenguaje de marcado estándar utilizado para estructurar el contenido del blog (perfil personal, listado de publicaciones, formulario de nuevo post).
- **CSS3:** Lenguaje de estilos ejecutado íntegramente en el navegador del cliente, sin ningún costo computacional para el servidor. Se optó por CSS3 puro, sin frameworks de terceros como Bootstrap, para evitar dependencias externas y mantener el proyecto autocontenido.

Esta combinación de tecnologías garantiza que el consumo de memoria del contenedor web en reposo sea cercano a cero, reservando los 128 MB disponibles para atender las peticiones entrantes.

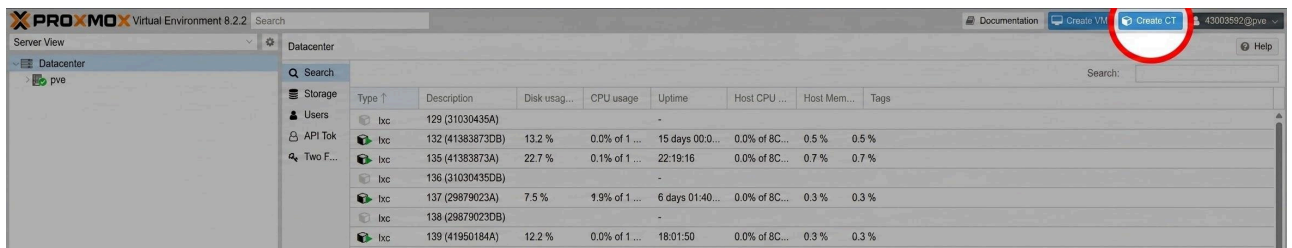
Paso a paso del desarrollo del blog

Creación y Configuración de la Infraestructura

Como primer paso, se ingresó a la plataforma Proxmox VE (nap.frt.utn.edu.ar) para crear los dos contenedores, siguiendo las reglas establecidas por la cátedra:

- **Contenedor Web (Frontend):**
 - **Hostname:** 43003592A
 - **ID asignado:** 177
 - **Dirección IP:** 172.16.90.177/24
- **Contenedor Base de Datos (Backend):**
 - **Hostname:** 43003592DB
 - **ID asignado:** 178
 - **Dirección IP:** 172.16.90.178/24

Para ambos contenedores, la Puerta de Enlace (Gateway) y el servidor DNS se configuraron apuntando a 172.16.90.1.



Creación del contenedor de frontend:

Create: LXC Container

General Template Disks CPU Memory Network DNS Confirm

Node: pve Resource Pool:

CT ID: 177 Password:

Hostname: 43003592A Confirm password:

Unprivileged container: ☒ SSH public key(s):

Nesting: ☒

Load SSH Key File

Create: LXC Container

General Template Disks CPU Memory Network DNS Confirm

Storage: local

Template: 12-standard_12.7-1_amd64.tar.zst

Name	For...	Size
debian-12-standard_12.7-1_amd64.tar.zst	tzst	126.52 MB
ubuntu-20.04-standard_20.04-1_amd64.tar.gz	tgz	214.20 MB
ubuntu-22.04-standard_22.04-1_amd64.tar.zst	tzst	129.82 MB
ubuntu-24.04-standard_24.04-2_amd64.tar.zst	tzst	141.59 MB
ubuntu-24.10-standard_24.10-1_amd64.tar.zst	tzst	143.47 MB

Siguiendo con los recursos establecidos por la cátedra de:

- Memoria 128 Mb
- 1 procesador
- Almacenamiento 8 Gb
- IP estática

Create: LXC Container ⓧ

General Template **Disks** CPU Memory Network DNS Confirm

rootfs 🗑

Storage: local-lvm
Disk size (GiB): 8

Create: LXC Container ⓧ

General Template Disks **CPU** Memory Network DNS Confirm

Cores: 1

Create: LXC Container ⓧ

General Template Disks CPU **Memory** Network DNS Confirm

Memory (MiB): 128
Swap (MiB): 128

Create: LXC Container ⓧ

General Template Disks CPU Memory **Network** DNS Confirm

Name: eth0
MAC address: auto
Bridge: vmb0
VLAN Tag: no VLAN
Firewall: ☒

IPv4: ☒ Static ☐ DHCP
IPv4/CIDR: 172.16.90.177/24
Gateway (IPv4): 172.16.90.1
IPv6: ☒ Static ☐ DHCP ☐ SLAAC
IPv6/CIDR: None
Gateway (IPv6):

Resumen Contenedor Frontend:

Create: LXC Container

General

Template

Disks

CPU

Memory

Network

DNS

Confirm

Key ↑	Value
cores	1
features	nesting=1
hostname	43003592A
memory	128
net0	name=eth0,bridge=vbr0,firewall=1,ip=172.16.90.177/24,gw=172.16.90.1
nodename	pve
ostemplate	local:vztmpl/debian-12-standard_12.7-1_amd64.tar.zst
pool	
rootfs	local-lvm:8
searchdomain	172.16.90.1
ssh-public-keys	
swap	128
unprivileged	1
vmid	177

☐ Start after created

Advanced ☐

Back

Finish

Creación del contenedor de Base de Datos:

Realizamos los mismos pasos que para la configuración del contenedor 177.

Create: LXC Container

General

Template

Disks

CPU

Memory

Network

DNS

Confirm

Key ↑	Value
cores	1
features	nesting=1
hostname	43003592DB
memory	128
net0	name=eth0,bridge=vbr0,firewall=1,ip=172.16.90.178/24,gw=172.16.90.1
nodename	pve
ostemplate	local:vztmpl/debian-12-standard_12.7-1_amd64.tar.zst
pool	
rootfs	local-lvm:8
searchdomain	172.16.90.1
ssh-public-keys	
swap	128
unprivileged	1
vmid	178

☐ Start after created

Advanced ☐

Back

Finish

Trabajo con el contenedor de base de datos (CT178)

Se inició el contenedor CT178 y se accedió a la consola, autenticándonos con el usuario `root` y la contraseña previamente establecida.

- Actualización e instalación de servicios

Se actualizaron los repositorios y se instaló el sistema gestor de bases de datos relacional.

```
apt update && apt install mariadb-server -y
```

- Configuración de red para conexiones remotas

Por defecto, MariaDB restringe las conexiones al entorno local (localhost). Para permitir la comunicación inter-contenedores con el servidor web (CT177), se editó el archivo de configuración principal

```
nano /etc/mysql/mariadb.conf.d/50-server.cnf
```



```
GNU nano 7.2 /etc/mysql/mariadb.conf.d/50-server.cnf *
#user = mysql
#pid-file = /run/mysqld/mysqld.pid
#basedir = /usr
#datadir = /var/lib/mysql
#tmpdir = /tmp

# Broken reverse DNS slows down connections considerably and name resolve is
# safe to skip if there are no "host by domain name" access grants
#skip-name-resolve

# Instead of skip-networking the default is now to listen only on
# localhost which is more compatible and is not less secure.
bind-address = 0.0.0.0

#
# * Fine Tuning
#
#key_buffer_size = 128M
#max_allowed_packet = 1G
```

Buscamos la línea que empieza con la directiva “bind-address” y vemos que por defecto tiene el valor asignado igual a 127.0.0.1. Modificamos ese valor por la IP de nuestro contenedor cliente o por 0.0.0.0 (“equivalente” a todas las IP).

-Registramos y reiniciamos el servicio:

```
systemctl restart mariadb
```

- Creación de esquema y credenciales de acceso

Se ejecutaron las siguientes sentencias SQL para crear la base de datos `blog_db`, aprovisionar el usuario `rociojaluf` y otorgar los privilegios globales necesarios para la correcta manipulación de los datos desde el servidor de origen remoto:

```
CREATE DATABASE blog_db;
CREATE USER 'rociojaluf'@'%'
IDENTIFIED BY 'rocio2026';
GRANT ALL PRIVILEGES ON blog_db.* TO 'rociojaluf'@'%';
FLUSH PRIVILEGES;
```

`MariaDB [(none)]>` nos indica que ya estamos adentro del motor:

```
MariaDB [(none)]> CREATE DATABASE blog_db;
Query OK, 1 row affected (0.000 sec)

MariaDB [(none)]> CREATE USER 'rociojaluf'@'%' IDENTIFIED BY 'rocio2026';
Query OK, 0 rows affected (0.259 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON blog_db.* TO 'rociojaluf'@'%';
Query OK, 0 rows affected (0.121 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.000 sec)

MariaDB [(none)]> EXIT;
Bye
root@43003592DB:~#
```

Con esto terminamos con la configuración del contenedor 178.

Trabajo con el contenedor del frontend (CT177)

Se inició el contenedor CT177 y se accedió a la consola, autenticándonos con el usuario `root` y la contraseña previamente establecida.

- **Actualización del sistema:** para asegurar que tenemos todos los paquetes actualizados.

```
apt update && apt upgrade -y
```

-**Instalación de Nginx y PHP:** se instaló el servidor web Nginx junto con el motor PHP-FPM y las extensiones necesarias para el funcionamiento del blog:

```
apt install nginx php8.2-fpm php8.2-mysqli php8.2-mbstring -y
```

Los paquetes instalados cumplen los siguientes roles:

- `nginx`: servidor web encargado de recibir y responder las solicitudes HTTP entrantes.
- `php8.2-fpm`: motor de ejecución PHP bajo el modelo FastCGI Process Manager, que permite procesar scripts PHP de forma eficiente y con bajo consumo de memoria.
- `php8.2-mysqli`: extensión que habilita la conexión entre PHP y el servidor MariaDB alojado en el CT178.
- `php8.2-mbstring`: extensión para el manejo correcto de cadenas de texto con caracteres multibyte, necesaria para la correcta visualización de tildes y caracteres especiales del español.

Despliegue del Frontend e Integración de Servicios

Una vez finalizada la configuración del entorno de base de datos (CT178) y la preparación del servidor web Nginx (CT177), se procedió con el despliegue de la capa de presentación y la lógica de aplicación.

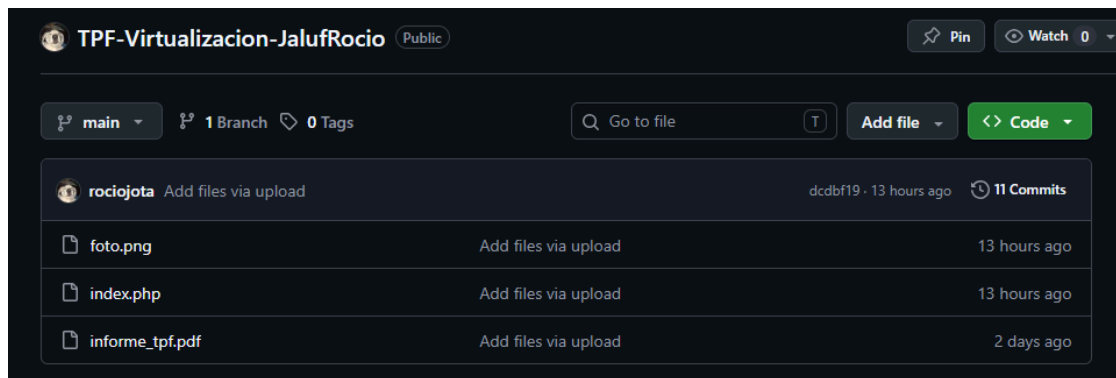
Se optó por implementar un flujo de trabajo basado en repositorios de control de versiones (GitHub). Esta decisión permite un manejo eficiente del código y facilita las actualizaciones continuas.

El proceso de despliegue se dividió en las siguientes fases:

1. **Creación del Repositorio Central:** Se generó un repositorio público en GitHub para alojar los recursos del proyecto. Se subieron los archivos desarrollados en el entorno local.
2. **Preparación del Entorno:** Dentro de la consola del CT177, se configuró el directorio raíz asignado por la cátedra (`/var/www/html/43003592`).
3. **Clonación y Despliegue:** Se ejecutó el comando `git clone` para descargar el proyecto directamente desde el repositorio hacia el contenedor web, asegurando que los archivos quedaran desplegados en la raíz del directorio. Para actualizaciones posteriores, se utilizó el comando `git pull`.

Los recursos transferidos e integrados en el servidor Nginx cumplen las siguientes funciones específicas:

- **index.php:** Archivo unificado que contiene la estructura semántica de la página (HTML5), la capa de estilos visuales (CSS3) y la lógica de backend (PHP 8.2). Este script es el responsable de interceptar la petición, establecer la conexión mediante la extensión `mysqli` hacia el contenedor de base de datos remoto (CT178) y renderizar la interfaz.
- **foto.png:** Recurso correspondiente a la foto de perfil. Fue optimizado en tamaño para garantizar una carga rápida.
- **informe_tpf.pdf:** Documento con el detalle de la arquitectura y despliegue del trabajo práctico final, alojado directamente en el servidor web para su descarga.



Link: <https://github.com/rociójota/TPF-Virtualizacion-JalufRocio.git>

Visualizamos que están correctamente en nuestro contenedor:

```
root@43003592A:~# ls -la /var/www/html/43003592
total 636
drwxr-xr-x 3 www-data www-data 4096 Jun 25 02:57 .
drwxr-xr-x 3 root root 4096 Jun 24 16:48 ..
drwxr-xr-x 8 www-data www-data 4096 Jun 25 02:21 .git
-rw-r--r-- 1 root root 444148 Jun 25 02:11 foto.png
-rw-r--r-- 1 root root 6600 Jun 25 02:09 index.php
-rw-r--r-- 1 root root 182867 Jun 24 16:59 informe_tpf.pdf
root@43003592A:~#
```

Tras la descarga del código fuente y los recursos estáticos verificamos el estado de recursos del contenedor:

```
root@43003592A:~# free -m
              total        used        free      shared  buff/cache   available
Mem:           128          30          28           2         71          97
Swap:          128           0          128
```

Gracias a la arquitectura orientada a eventos, el consumo real de RAM (*used*) se mantiene en apenas **30 MB**. Esto certifica que el contenedor opera con fluidez, conservando un margen de disponibilidad holgado de 97 MB libres respecto al límite máximo de 128 MB.

```
root@43003592A:~# df -h
Filesystem              Size  Used Avail Use% Mounted on
/dev/mapper/pve-vm--177--disk--0 7.8G 931M 6.5G  13% /
none                    492K  4.0K 488K   1% /dev
udev                   3.8G   0 3.8G   0% /dev/tty
tmpfs                  3.9G   0 3.9G   0% /dev/shm
tmpfs                  1.6G  88K 1.6G   1% /run
tmpfs                  5.0M   0 5.0M   0% /run/lock
```

La evaluación demuestra la viabilidad de la restricción de 8 GB. El sistema operativo Debian base, las dependencias instaladas y el repositorio completo del proyecto ocupan únicamente **931 MB**. Esto deja 6.5 GB completamente disponibles para el crecimiento del proyecto.

Visualización del Blog desplegado



Blog personal Rocío Jaluf

- **DNI:** 43003592
- **Carrera:** Ingeniería en Sistemas de Información
- **Institución:** UTN - FRT
- **Materia:** Virtualización y consolidación de servidores
- **Hobby:** Correr

Publicar nuevos datos al blog

Escribe aquí tu actualización...

Agregar dato

Documentación trabajo práctico final

 Descargar el informe en PDF

Validación de Persistencia y Gestión de Datos

Para certificar la correcta integración entre la capa de presentación (Frontend - CT177) y la capa de datos (Backend - CT178), se ejecutó una consulta para confirmar que la información ingresada a través de la interfaz web es recibida en la DB:

Publicar nuevos datos al blog

Agregar dato



Blog personal Rocío Jaluf

- **DNI:** 43003592
- **Carrera:** Ingeniería en Sistemas de Información
- **Institución:** UTN - FRT
- **Materia:** Virtualización y consolidación de servidores
- **Hobby:** Correr
- **Color Favorito:** Celeste

Ingresamos al contenedor de DB y hacemos la consulta en la tabla del blog:

```
MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| blog_db  |
| information_schema |
+-----+
2 rows in set (0.000 sec)

MariaDB [(none)]> USE blog_db
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [blog_db]> SHOW tables;
+-----+
| Tables_in_blog_db |
+-----+
| datos_personales |
+-----+
1 row in set (0.000 sec)
```

```
MariaDB [blog_db]> select * from datos_personales;
+----+-----+
| id | contenido |
+----+-----+
| 1  | Hobby: Correr |
| 2  | Color Favorito: Celeste |
+----+-----+
2 rows in set (0.020 sec)
```

Como se evidencia en las capturas anteriores, el registro generado desde el navegador del cliente fue insertado con éxito en la tabla **datos_personales** de **blog_db**.

Conclusión

El desarrollo e implementación del Trabajo Práctico Final permitió consolidar de manera práctica los conceptos de virtualización, administración de servidores Linux y arquitectura de aplicaciones web.

La utilización de la plataforma Proxmox VE y la tecnología de contenedores LXC demostró ser una solución altamente eficiente, permitiendo el despliegue de una infraestructura funcional bajo restricciones de recursos estrictas.

El foco del proyecto radicó en la selección adecuada de herramientas. La implementación de Nginx como servidor web orientado a eventos, en conjunto con PHP-FPM, garantizó un consumo de memoria mínimo.

Asimismo, la configuración de MariaDB como repositorio de datos centralizado, asegurando la comunicación intercontenedores, demostró la viabilidad de arquitecturas distribuidas en entornos virtualizados.

Finalmente, la integración de un flujo de despliegue mediante repositorios gratuitos como GitHub y la validación exitosa de la persistencia de datos confirma el cumplimiento de los requisitos funcionales y técnicos exigidos por la cátedra.